

RIGEL · SPEC EDITOR

Luau Scripting Guide

Write scripts for the objects you place, and run them on every player's machine.

Contents

- 1 What a script is and where it runs
- 2 Attaching a script in the editor
- 3 Saving scripts with levels
- 4 Anatomy of a script
- 5 Core globals and patterns
- 6 Components and how to find their methods
- 7 Worked examples from this kit
- 8 Fun scripts to drop in
- 9 Worked examples: components and the game's scripts
- 10 Setting up VS Code
- 11 Troubleshooting
- 12 Limitations
- A Appendix: component and class reference
- B Appendix: global tables

1 What a script is and where it runs

A **Luau script** is a small program attached to an object in the station. Orion Drift's own game modes are written this way. TKBGolf's `Course.luau`, for example, sits on a plain Cube object and runs the whole mini-golf course. The Spec Editor lets you attach your own scripts to objects you place, using the same mechanism the game uses.



It runs everywhere

This is the most important fact about scripts. The same source runs **on every machine that has the object**: the dedicated server, your own client, and every other player's client, including players with no mods. The test that confirmed this was a one-line script:

```
log("RIGEL LUAU OK " .. tostring(20 + 22))
```

and the line `[RigelTest.luau]: RIGEL LUAU OK 42` appeared in the logs of the server, the editor client and an unmodded player's client.

What that means for your code

- **Side effects happen once per machine.** A `log` call prints once in each log. An action that changes shared game state (scoring, spawning, teleporting) may run on several machines at once. The game scripts handle this by listening to *server* events for authoritative work (`onOverlapByPlayerServer`, `serverEventData`) and to *client-side* events for per-player feedback (`onOverlapByPlayerClientside`).
- **"The local player" differs per machine.** `VRPawn.getLocalPlayerIndex()` returns a different value on each client. The golf course uses exactly that to give every player their own personal ball.
- **Variables are not shared.** Each machine has its own copy of every variable. To tell other machines something, the game scripts use components such as `EventComponent` (`triggerEventWithData` → `serverEventData`).

Scope of this guide

The kit is built from three sources: the game's own scripts (in `game-scripts/`), the game's class dump (a Dumper-7 C++ SDK of build 22284) and tests run in Rigel. Where something has only been inferred, the guide says so. Chapter 12 lists the limits.

2 Attaching a script in the editor

Scripts are ordinary `.luau` files in your **RigelScripts** folder (`Documents\RigelScripts`). The editor creates it the first time and copies this kit into it (types, examples, VS Code settings), so autocomplete works straight away. You write scripts in VS Code, not inside the editor.

- 1 **Open the editor** with **F12**, and place or select an object.
- 2 In **Details**, open **Luau script** and click **Open folder in VS Code**.
- 3 **Write the script** as a new file, e.g. `DoorLogic.luau`, or copy one from `examples/`. The file name is the script's name: it is stored as `Scripts/DoorLogic.luau`, and `[DoorLogic.luau]` is what you'll see in logs.
- 4 Back in the editor, pick the file in the list (press **Refresh** if it's new) and click **Attach to this object**.
- 5 **Wire its slots** (chapter 4): under the script in **Details** → **Game data**, click a slot and then click the object it should use, in the viewport or the Outliner. You can also drag the object onto the slot, or pick from the slot's ▼ list.
- 6 **Keep editing in VS Code.** Every time you save the file, the editor re-sends it, and every object running that script is rebuilt with the new code, on every machine.

What happens when you attach

The server stores your source in the object's gamemode under `Scripts/<name>.luau`, then **rebuilds the object** with a component entry whose id and script are both `<name>.luau`. The game attaches its own scripts in exactly this way. Because the object is rebuilt:

- the old copy of the object (and any script that was running on it) goes away, and the new one runs from the top, `BeginPlay` included;
- anything your script had set up (timers, listeners, variables) starts fresh;
- attaching the **same file** again replaces the script, and a different file adds another script entry.

Scripts already on an object

Selecting any object, including the station's own, lists its scripts in **Game data** as `Script <name>.luau`. **Replace** swaps in a file from your folder, and **Remove** takes the script off an object you placed (the object is rebuilt without it). On objects you placed this happens right away. On the station's objects, players who join from then on get the new code.

Borrowing from the game's scripts

Copy the game's own scripts into the folder (examples) copies the station's scripts (golf, Scraprun, Tackleball, ...) into `RigelScripts\game-scripts`, where you can read them in VS Code.

3 Saving scripts with levels

Scripts are part of the level. When you save a level in the editor's **Levels** tab, every script attached to its objects is saved with it. When the level is loaded again, the scripts are **re-attached** to their objects, so the objects are rebuilt and the scripts start from the top on every machine.

- Save the level after attaching or changing a script. The attached script lives on the running server, and only the saved level brings it back after a restart.
- Your RigelScripts files stay the source of truth. A level stores a copy of each script and its slot wiring.
- Loading a level re-runs `BeginPlay` for its scripts. Write `BeginPlay` so that running it again on a fresh object is harmless.

Workflow that works well

Attach once, then just edit and save in VS Code: each save re-sends the script. Read the log, repeat. When it behaves, save the level.

4 Anatomy of a script

Every game script follows the same shape. Here is the start of TKBGolf's course script, cut down:

game-scripts/TKBGolf_Course.luau, lines 1-12

```
--!strict
-- External Dependencies

local MainEventRepeater: EventComponent = nil
local DataTable: DataTableComponent = nil

local Cup_1: GolfCupComponent = nil
local Cup_2: GolfCupComponent = nil
local Cup_3: GolfCupComponent = nil
local Cup_4: GolfCupComponent = nil
local Cup_5: GolfCupComponent = nil
```

and here is how a script built in the same shape reads, top to bottom:

the parts of a script

```
--!strict -- 1. mode line

-- External Dependencies -- 2. components this script talks to
local Button: BasicButtonComponent = nil
local Door: PhysicalComponent = nil
-- End External Dependencies

local doorOpen = false -- 3. your own state

local function toggle() -- 4. helpers
    doorOpen = not doorOpen
    if doorOpen then Door:hideLua() else Door:showLua() end
end

function BeginPlay() -- 5. lifecycle entry point
    Button.onButtonPressEvent.Listen(toggle)
end

function EndPlay() -- 6. optional clean-up
end
```

1. `--!strict`

The first line picks the Luau type-checking mode. The golf scripts use `--!strict`, and the gamemode scripts (Scraprun, Tackleball) have no mode line. The mode matters to the type checker in VS Code, and `--!strict` gets you the most help from it. This kit's `.luaurc` turns strict mode on for every file.

2. The External Dependencies block

Typed locals set to `nil`, one per component the script uses. **The game fills these in before** `BeginPlay` from the object's `references` wiring: a reference named `Cup_1` becomes the value of `local Cup_1`. The type after the colon is the component's Luau type (`GolfCupComponent`, `PhysicalComponent`, ...), which is also what gives you autocomplete.

Slots: wire them by dragging

Each dependency local is a **slot**. Under the script in **Details** → **Game data**, the editor lists every slot (`Switch` (`ToggleableComponent`), `Target` (`PhysicalComponent`), ...). Click a slot and then click the object (in the viewport or the Outliner). Dragging an object onto the slot also works, and the ▼ button lists every object you placed that fits. **X** clears the slot, and **Esc** cancels a pick. The object must have a component of that type; if it doesn't, the editor explains which components it does have. Objects are moved into the scripted object's gamemode area if needed. The script is rebuilt with the new wiring, and the wiring is saved with the level. Still check for `nil` in `BeginPlay`, so an unwired slot logs a hint instead of an error.

3–4. State and helpers

Plain Luau. Prefer `local` variables and functions. The game scripts also use globals freely (`currentState = ...`). That works, but everything in the script can then change them.

5–6. Lifecycle

When	What runs
Script loaded	All top-level code, in order. The dependency locals are still <code>nil</code> here.
Object ready	<code>function BeginPlay()</code> , if you define it. Every game script does its setup here: hooking events and setting the initial state.
Object/gamemode torn down	<code>function EndPlay()</code> , if defined (Scraprun clears its tables here). Attaching a new version of your script rebuilds the object, which ends the old one.
Later	Only what you scheduled: event listeners and timers/promises.

`BeginPlay` and `EndPlay` must be **global** functions (`function BeginPlay()`, not `local function`) so that the game can find them. No per-frame `Tick` function appears in any game script, so for anything repeating use timers (chapter 5).

5 Core globals and patterns

Logging

```
log("score is", score)           -- LogLuau: Display: [MyScript.luau]: score is 3
warn("something odd")            -- LogLuau: Warning: [MyScript.luau]: something odd
Logging.warn("Category", value)  -- used by THKGolf's ServerGolfLogic
```

`log` takes any number of values. Build strings with `..` or backtick interpolation (``hole {i} done``), as the game scripts do.

Timers: `LuauClock`

Call	Meaning
<code>LuauClock.timeout(seconds)</code>	A promise that resolves after <i>seconds</i> . <code>timeout(0)</code> means "next moment".
<code>LuauClock.getTime()</code>	Current time in seconds. Subtract two readings to measure a duration.
<code>LuauClock.createTimer(interval, fn)</code>	Found in the class dump (<code>ULuauClock::CreateTimer</code>), but no game script uses it. Untested.

```
LuauClock.timeout(5):andThen(function()
    Cup:setEnableArrowIndicator(false)    -- hide the arrow 5 s later (ClubGolf)
end)
```

Promises

Asynchronous work comes back as a promise. The game scripts use three shapes:

```
-- a) chain: run a function when it resolves
Tee:spawnBallWithParameters(params):andThen(function(Disc: DiscEntity)
    Disc:setEnableHUDIndicator(true)
end)

-- b) straight-line: :await() pauses this function until the promise resolves
Promise.new(function()
    ResetTrigger:luaEnable()
    LuauClock.timeout(0.25):await()
    ResetTrigger:luaDisable()
end)

-- c) whichever finishes first
Promise.race({ LuauClock.timeout(45), earlyOut }):andThen(function() ... end)
```

`:await()` only makes sense inside `Promise.new(function() ... end)`, which runs its function as its own coroutine. Scraprun uses this for every "do X, wait, do Y" sequence. `Promise.resolve(v)`, `Promise.try(fn)` and `:expect()` appear in commented-out code in the golf scripts.

Events

Components expose C++ multicast delegates as events. Listen with a **dot**:

```
Disc.onGoToSleep.Listen(function()
    Disc:addSmoothOffset(Disc:getGravityDirection() * -10)
end)
Cup.onBallInCup_Multicast.Listen(function(Cup: GolfCupComponent, Score: number, Disc: DiscEntity)
    ...
end)
BasicTimer.finished.Listen(BasicTimerFinished)    -- pass a named function
```

The callback gets the delegate's arguments, in order: `TMulticastInlineDelegate<void(UGolfCupComponent* Cup, int32 Score, ADiscEntity* Entity)>` becomes `(GolfCupComponent, number, DiscEntity)`. The colon form `Player.onBeforeDestroy:Listen(fn)` also appears in a game script, and the definitions accept both. Event names are lowerCamel (`onGoToSleep`), but the game scripts also use the original C++ spelling (`BlueprintOnEnabled`, `TeamSizeIncreased`), so both spellings are in the definitions.

Vectors

```
local v = Vector.new(0, 0, 1)
local d = disc:getDiscPosition() - VRPawn.getPositionByID(playerId) -- Vector - Vector
local dist = math.sqrt(d.x * d.x + d.y * d.y + d.z * d.z) -- .x .y .z
local dir = d / dist -- Vector / number
ball:setVelocity(dir * 3500 + Vector.new(0, 0, 200)) -- *, +
```

Units are Unreal units (centimetres). `Vec3.new` is an older spelling used once in the Tackleball script. C++ `FRotator` parameters take a `Vector` of (pitch, yaw, roll), as `requestTrainingBallAtLocation(Location, Rotation)` shows.

Other globals

Global	What it's for (as used in the game scripts)
<code>VRPawn.getLocalPlayerIndex()</code>	Index of this machine's player.
<code>VRPawn.getPlayerByID(id)</code> , <code>getPlayerName(id)</code> , <code>getTeamIndexByID(id)</code> , <code>getPositionByID(id)</code>	Look up players by index. A name of "None" means unknown.
<code>Quests.event(player, id)</code> , <code>Quests.eventNumber(player, id, n)</code> , <code>Quests.complete(player, questId)</code>	Quest/stat progress. The golf scripts pass <code>-1</code> from client-side handlers.
<code>Gamemode.getNumberConfigVariable(name)</code> , <code>getBoolConfigVariable</code> , <code>startGame()</code> , <code>stopGame()</code> , <code>setTeamScore(t, n)</code> , <code>onConfigChanged</code>	The gamemode the script belongs to. Only the gamemode scripts use it, and it's untested from a sandbox object's script.
<code>BallSpawnParameters.new{...}</code> , <code>GoalInfo.new{...}</code>	Build C++ structs from a table with lowerCamel keys.
<code>LinearColor.new(r, g, b, a)</code> , <code>CellStyle.new</code> , <code>LayoutSettings.new</code> , <code>LayoutGap.new</code> , <code>StyleOverRange.new</code> , <code>Enum.TextJustify</code>	Scoreboard styling (<code>DataTableComponent</code>).
<code>require("Name")</code>	Loads another script from the same gamemode (TKBGolf's <code>ServerCourseLogic</code> requires <code>ServerGolfLogic</code>). Not tested with editor-attached scripts, and VS Code can't resolve it.

6 Components and how to find their methods

A **component** is a C++ class deriving from `ULuauBehavior`. Buttons, triggers, timers, ball spawners, goals, scoreboards and toggles are all components. Scripts reach them through the External Dependencies block and then call their methods and listen to their events.

The naming rule

In C++ (class dump)	In Luau
<code>class UGolfCupComponent : public ULuauBehavior</code>	type <code>GolfCupComponent</code> (drop the <code>U</code>)
<code>class ADiscEntity : public AActor</code>	type <code>DiscEntity</code> (drop the <code>A</code>)
<code>void SetCupVisibility(bool Show);</code>	<code>cup:setCupVisibility(show)</code> (lowerCamel, colon call)
<code>static int32 GetLocalPlayerIndex(UWorld* World);</code>	<code>VRPawn.getLocalPlayerIndex()</code> (dot call, world supplied for you)
<code>TMulticastInlineDelegate<void> OnGoToSleep;</code>	<code>disc.onGoToSleep.Listen(function() end)</code>
<code>void Client_EmitStatEvent(const FString&, int32);</code>	<code>pawn:client_EmitStatEvent("Stat", 1)</code> (only the first letter changes)

The generated definitions

`generate_defs.py` reads the class dump (`HalcyonA2/HalcyonA2/gamesdk/22284/SDK`) and writes `types/rigel.d.luau`, a `luau-lsp` definition file. The current build covers **49 classes** (36 of them `ULuauBehavior` components) with **524 methods**, **119 events**, **239 properties**, **16 static functions** and **19 structs**. Each method carries its original C++ declaration as a comment:

`types/rigel.d.luau` (excerpt)

```
declare extern type GolfCupComponent extends LuauBehavior with
  -- C++: void SetCupVisibility(bool Show);
  function setCupVisibility(self, show: boolean): ()
  ...
  onBallInCup_Multicast: Event<(GolfCupComponent, number, DiscEntity)>
end
```

Type mapping

C++	Luau
<code>bool</code>	<code>boolean</code>
<code>int32</code> , <code>float</code> , <code>double</code> , <code>uint8</code> ...	<code>number</code> (the game scripts also write <code>int</code> , an alias)
<code>FString</code> , <code>FName</code> , <code>FText</code>	<code>string</code>
<code>FVector</code> , <code>FRotator</code>	<code>Vector</code>
<code>UFoo*</code> / <code>AFoo*</code>	<code>Foo</code> when it is in the definitions, otherwise <code>any</code>
<code>TArray<T></code>	<code>{T}</code>
<code>FLuaRef</code>	<code>Ref</code> (any Luau table)
game structs (<code>FGoalInfo</code> , ...)	a type with lowerCamel fields (<code>goalInfo.scoringTeam</code>)
out-parameters (<code>T* Out</code>)	returned as extra results
anything else	<code>any</code>

Blueprint-backed types

Some types in the game scripts aren't C++ components. They're Blueprint actors with a Luau bridge: `GolfTeeComponent`, `TrapSpinnerComponent`, `SlidingPlatformComponent`, `GameStateManagerBlueprintComponent` and others. The generator maps each one to the Blueprint class whose functions match the calls in the scripts (for example `GolfTeeComponent` → `BP_GolfTee_C`, which has `onManualStart` and `Set_Enable_Arrow`). Blueprint functions are listed under several spellings, because the scripts call them as written (`GameStateManagerBP:PlaySaveEvent()`), in lowerCamel (`setIsRestartable`) and without underscores (`setEnableArrow`). These mappings are inferred from the scripts.

Finding what you need

1. In VS Code, type the variable name and `:` (methods) or `.` (events and properties).
2. *Go to Definition* (F12) on a type name opens `rigel.d.luau` at that class, with the C++ comments.
3. Search `game-scripts/` for the method name. A method the game itself calls is much more likely to work than one that only appears in the dump.
4. Appendix A lists every class in the definitions with its key methods and events.

7 Worked examples from this kit

All six files in `examples/` type-check with no errors against the definitions (`luau-lsp analyze`, strict mode). Examples 1–3 only use globals, so they can run on any object you place. Examples 4–6 need their slots wired (chapter 4).

Example 1: hello

examples/01_hello.luau

```
--!strict
-- 01_hello.luau
-- The smallest useful script. Attach it to any sandbox object.
-- Every machine that has the object (the server, the editor, every player) runs it,
-- so you will see the line once in EACH machine's log:
--   LogLuau: Display: [01_hello.luau]: Hello from player 3 -- 20 + 22 = 42

-- Code at the top level runs as soon as the script is loaded.
local answer = 20 + 22

-- BeginPlay is called by the game once the script's object is ready.
function BeginPlay()
    -- VRPawn.getLocalPlayerIndex() is the index of the player on THIS machine.
    -- (On the dedicated server there is no local player, so expect a placeholder value there.)
    local me = VRPawn.getLocalPlayerIndex()
    log("Hello from player " .. tostring(me) .. " -- 20 + 22 = " .. tostring(answer))
end

-- EndPlay is called when the object or its gamemode is torn down (for example when the
-- object is rebuilt because you attached a new version of this script).
function EndPlay()
    log("Goodbye")
end
```

--!strict	Full type checking in VS Code.
local answer = 20 + 22	Top-level code runs as soon as the script loads, on every machine.
function BeginPlay()	Global, so the game can call it once the object is ready.
VRPawn.getLocalPlayerIndex()	A static UFunction, called with a dot. Each client gets its own index.
log("Hello from player ")	One line per machine in LogLuau .
function EndPlay()	Runs when the object is rebuilt or removed.

Example 2: repeating timer

examples/02_repeating_timer.luau

```
--!strict
-- 02_repeating_timer.luau
-- A timer loop: log a heartbeat every 2 seconds, and stop after 5 beats.
--
-- LuauClock.timeout(seconds) returns a promise that resolves after that many seconds.
-- :andThen(fn) runs fn when it resolves. To repeat, schedule the next timeout from inside fn.

local INTERVAL = 2      -- seconds between beats
local MAX_BEATS = 5

local beats = 0
local startedAt = 0

local function beat()
    beats += 1
    local elapsed = LuauClock.getTime() - startedAt
    log(`beat {beats} at {math.floor(elapsed * 10 + 0.5) / 10} s`)

    if beats < MAX_BEATS then
        LuauClock.timeout(INTERVAL):andThen(beat)
    else
        log("done")
    end
end

function BeginPlay()
    startedAt = LuauClock.getTime()
    LuauClock.timeout(INTERVAL):andThen(beat)
end
```

<code>LuauClock.getTime()</code>	Record the start time so each beat can report the elapsed time.
<code>LuauClock.timeout(INTERVAL):andThen(beat)</code>	Schedule <code>beat</code> . Inside <code>beat</code> the same call schedules the next one, which makes a loop.
<code>if beats < MAX_BEATS then ... else log("done")</code>	Always give a loop a way to stop, or it keeps running on every machine for as long as the object exists.
<code>`beat {beats} at ...`</code>	Backtick string interpolation, as used in the game scripts.

Example 3: promises

examples/03_promise_sequence.luau

```
--!strict
-- 03_promise_sequence.luau
-- Two ways to wait: chaining with :andThen, and writing straight-line code with :await().
-- Both patterns are taken from the game's own scripts (TKBGolf, ScraprunPrime).

-- Straight-line style: inside Promise.new(function() ... end) you may call :await(),
-- which pauses only this function, not the game.
local function countdown(from: number)
    Promise.new(function()
        for n = from, 1, -1 do
            log("T-minus " .. n)
            LuauClock.timeout(1):await()
        end
        log("Go!")
    end)
end

-- Racing a timeout against something that may happen first (TKBGolf uses this to hide an
-- arrow after 45 seconds OR as soon as the player starts another hole).
local cancel = (function() end) :: () -> () -- a cast: a typed top-level local would be an editor property

local function waitForSignalOrTimeout(seconds: number)
    local signalled = Promise.new(function(resolve: (number) -> (), _reject: (any) -> ())
        cancel = function()
            resolve(1)
        end
    end)

    Promise.race({ LuauClock.timeout(seconds), signalled }):andThen(function()
        cancel = function() end
        log("either the timeout fired or cancel() was called")
    end)
end

function BeginPlay()
    countdown(3)
    waitForSignalOrTimeout(10)
    -- Something else in your script could call cancel() to finish early:
    LuauClock.timeout(4):andThen(function()
        cancel()
    end)
end
```

<code>Promise.new(function() ... end)</code>	Runs the function as its own coroutine, so <code>:await()</code> can pause it.
<code>LuauClock.timeout(1):await()</code>	Wait one second, then continue with the next line.
<code>Promise.new(function(resolve, _reject) ... end)</code>	A promise you resolve yourself. It keeps <code>resolve</code> in <code>cancel</code> for later.
<code>Promise.race({ timeout, signalled })</code>	Resolves when the first of the two resolves. TKBGolf uses this to hide an arrow early.

8 Fun scripts to drop in

Ready-made scripts with slots, in `examples/` (and in your RigelScripts folder). Attach one and drag objects onto its slots.

Light switch: show and hide an object

examples/07_light_switch.luau

```
--!strict
-- 07_light_switch.luau
-- A light switch that makes another object appear and disappear.
--
-- How to use it:
-- 1. Place a Light Switch (or any switch) and the object it should control (a cube, a wall, a platform...).
-- 2. Select the switch, open Details > Luau script, pick LightSwitch and click "Attach to this object".
-- 3. In Details > Game data, under "Script LightSwitch.luau", drag the switch itself from the Outliner onto
--    the Switch slot, and the object onto the Target slot.
-- Flip the switch: the target shows when it's on and hides (and stops blocking players) when it's off.
-- Good targets are things whose look IS their physical part: walls, platforms, cubes, ramps, meshes.
-- A few gameplay objects keep their visible model separate (a golf tee, for one) and won't disappear.

-- External Dependencies (slots: drag objects onto these in the editor)
local Switch: ToggleableComponent = nil
local Target: PhysicalComponent = nil
-- End External Dependencies

local function apply(on: boolean)
    if on then
        Target:showLua()
        Target:setDefaultCollision()
    else
        Target:hideLua()
        Target:disableCollisionLua()
    end
end

function BeginPlay()
    local missing = {}
    if (Switch :: ToggleableComponent?) == nil then table.insert(missing, "Switch") end
    if (Target :: PhysicalComponent?) == nil then table.insert(missing, "Target") end
    if #missing > 0 then
        log("LightSwitch: pick objects for these slots in the editor (Details > Game data): " ..
            table.concat(missing, ", "))
        return
    end
    Switch.blueprintOnEnabled.Listen(function() apply(true) end)
    Switch.blueprintOnDisabled.Listen(function() apply(false) end)
    apply(Switch.isEnabled)
    log("LightSwitch ready")
end
```

Vanishing platform

examples/08_vanishing_platform.luau

```
--!strict
-- 08_vanishing_platform.luau
-- A platform that disappears a moment after a player touches it, then comes back. Great for parkour.
--
-- Attach it to the platform (a cube or any mesh) and drag that same object onto the Platform slot.

-- External Dependencies
local Platform: PhysicalComponent = nil
-- End External Dependencies

local VANISH_AFTER = 0.6 -- seconds after the first touch
local COME_BACK_AFTER = 3 -- seconds hidden

local busy = false

function BeginPlay()
    if (Platform :: PhysicalComponent?) == nil then
        log("VanishingPlatform: pick the platform for the Platform slot in the editor")
        return
    end
    Platform.onOverlapByPlayerSimple.Listen(function()
        if busy then return end
        busy = true
        LuauClock.timeout(VANISH_AFTER):andThen(function()
            Platform:hideLua()
            Platform:disableCollisionLua()
            return LuauClock.timeout(COME_BACK_AFTER)
        end):andThen(function()
            Platform:showLua()
            Platform:setDefaultCollision()
            busy = false
        end)
    end)
end
```

Blinker

examples/09_blinker.luau

```
--!strict
-- 09_blinker.luau
-- Makes an object blink on and off forever: a timing puzzle for parkour, or just a flashing sign.
--
-- Attach it to anything and drag the object that should blink onto the Target slot.

-- External Dependencies
local Target: PhysicalComponent = nil
-- End External Dependencies

local ON_SECONDS = 2
local OFF_SECONDS = 1.5

local function cycle()
    Target:showLua()
    Target:setDefaultCollision()
    LuauClock.timeout(ON_SECONDS):andThen(function()
        Target:hideLua()
        Target:disableCollisionLua()
        return LuauClock.timeout(OFF_SECONDS)
    end):andThen(cycle)
end

function BeginPlay()
    if (Target :: PhysicalComponent?) == nil then
        log("Blinker: pick an object for the Target slot in the editor")
        return
    end
    cycle()
end
```

Secret door

examples/10_secret_door.luau

```
--!strict
-- 10_secret_door.luau
-- A secret door: a button opens it for a few seconds, then it closes again by itself.
--
-- Attach it to the button, then drag the button onto the Button slot and the wall/door onto the Door slot.

-- External Dependencies
local Button: BasicButtonComponent = nil
local Door: PhysicalComponent = nil
-- End External Dependencies

local OPEN_SECONDS = 5
local open = false

function BeginPlay()
    if (Button :: BasicButtonComponent?) == nil or (Door :: PhysicalComponent?) == nil then
        log("SecretDoor: pick objects for the Button and Door slots in the editor")
        return
    end
    Button.onButtonPressEvent.Listen(function()
        if open then return end
        open = true
        Door:hideLua()
        Door:disableCollisionLua()
        LuauClock.timeout(OPEN_SECONDS):andThen(function()
            Door:showLua()
            Door:setDefaultCollision()
            open = false
        end)
    end)
end
```

9 Worked examples: components and the game's scripts

Example 4: a button that toggles a door

examples/04_button_toggles_door.lua

```
--!strict
-- 04_button_toggles_door.lua
-- React to an event from a referenced component and drive another one.
--
-- IMPORTANT: the two locals in the External Dependencies block are filled in by the game from
-- the object's 'references' wiring, matching by name: in the editor, Details > Game data > the
-- script's slots (click a slot, then click the object). Until wired they are nil; the nil checks
-- below keep the script from erroring in that case.

-- External Dependencies
local Button: BasicButtonComponent = nil
local Door: PhysicalComponent = nil
-- End External Dependencies

local doorOpen = false

local function setDoor(open: boolean)
    doorOpen = open
    if open then
        Door:hideLua()           -- make it invisible ...
        Door:disableCollisionLua() -- ... and let players walk through
    else
        Door:showLua()
        Door:setDefaultCollision()
    end
    log("door " .. (if open then "opened" else "closed"))
end

function BeginPlay()
    if (Button :: BasicButtonComponent?) == nil or (Door :: PhysicalComponent?) == nil then
        log("Button/Door references are not wired; nothing to do")
        return
    end

    -- Events are listened to with a DOT: component.eventName.Listen(function(...) end)
    Button.onButtonPressEvent.Listen(function()
        setDoor(not doorOpen)
    end)

    setDoor(false)
end
```

local Button: BasicButtonComponent = nil

Filled from a reference named `Button`. The type gives you `Button.onButtonPressEvent` and `Button.luaEnableButton()`.

(Button :: BasicButtonComponent?) == nil

The cast to an optional type lets strict mode accept a `nil` comparison. At runtime it's a plain nil check.

Door:hideLua() / disableCollisionLua()

`PhysicalComponent` methods. Scraprun opens its forcefields with this exact pair.

Button.onButtonPressEvent.Listen(...)

Dot + `Listen`. Scraprun hooks every trap button this way.

Example 5: a trigger zone

examples/05_trigger_zone.lua

```
--!strict
-- 05_trigger_zone.lua
-- A PhysicalComponent used as a trigger volume: greet players who walk into it,
-- and hand out a quest event.
--
-- As in 04, `Zone` comes from the object's references wiring: Details > Game data > the script's
-- slots (click Zone, then click the object) -- or the MCP tool set_script_reference.

-- External Dependencies
local Zone: PhysicalComponent = nil
-- End External Dependencies

local visits = {} :: { [number]: number } -- a cast, not a typed local: typed top-level locals are editor
properties

function BeginPlay()
    if (Zone :: PhysicalComponent?) == nil then
        log("Zone reference is not wired")
        return
    end

    -- Make the volume a trigger (overlaps fire, nothing blocks).
    Zone:setTriggerCollision()

    -- Fires on the server with the index of the player who entered.
    Zone.onOverlapByPlayerServer.Listen(function(playerIndex: number)
        visits[playerIndex] = (visits[playerIndex] or 0) + 1
        local name = VRPawn.getPlayerName(playerIndex)
        local team = VRPawn.getTeamIndexByID(playerIndex)
        log(`${name} (team {team}) entered the zone, visit #{visits[playerIndex]}`)
    end)

    -- Fires on the machine of the player who entered (no argument): good for per-player
    -- effects such as quest progress. -1 means "the local player", as in the golf scripts.
    Zone.onOverlapByPlayerClientside.Listen(function()
        Quests.event(-1, "MyZone_Entered")
    end)
end
```

`onOverlapByPlayerServer` passes the player index, and `onOverlapByPlayerClientside` passes nothing and fires on the entering player's own machine. That split is the one `TKBGolf__Course.lua` uses to start a hole for the player who walked in.

Example 6: vectors and balls

examples/06_vectors_and_balls.luau

```
--!strict
-- 06_vectors_and_balls.luau
-- Vector maths plus a ball spawner: spawn a personal ball for the local player, then
-- launch it straight up whenever it comes to rest. Mirrors TKBGolf's Course.luau.
--
-- `Spawner` comes from the object's references wiring (not configurable in the editor yet).

-- External Dependencies
local Spawner: BallSpawnerComponent = nil
-- End External Dependencies

-- Vectors support + - * / and have .x .y .z
local function length(v: Vector): number
    return math.sqrt(v.x * v.x + v.y * v.y + v.z * v.z)
end

local function launch(ball: DiscEntity)
    local up = ball:getGravityDirection() * -1 -- opposite of gravity
    ball:setVelocity(up * 1500) -- Unreal units: cm/s
    log(`launched; speed now {math.floor(length(ball:getDiscVelocity()))}`)
end

function BeginPlay()
    if (Spawner :: BallSpawnerComponent?) == nil then
        log("Spawner reference is not wired")
        return
    end

    local params = BallSpawnParameters.new({
        position = Spawner:getActorLocationLuau() + Vector.new(0, 0, 100),
        isPersonalBall = true,
        targetPlayerIndex = VRPawn.getLocalPlayerIndex(),
    })

    -- spawnBallWithParameters returns a promise that resolves with the ball on this machine.
    Spawner:spawnBallWithParameters(params):andThen(function(ball: DiscEntity)
        ball:setEnableHUDIndicator(true)
        ball.onGoToSleep.Listen(function()
            launch(ball)
        end)
    end)
end
```

The spawn call follows TKBGolf's course script. `spawnBallWithParameters` is declared in C++ as returning `ADiscEntity*`, but in Luau it returns a **promise** of the ball (the game script chains `:andThen(function(Disc: DiscEntity) ...)`). The definitions include that correction.

From the game: starting a hole (TKBGolf)

game-scripts/TKBGolf_Course.luau, lines 130-151

```
local Parameters = BallSpawnParameters.new({
    position = Vector.new(1, 2, 3),
    isPersonalBall = true,
    targetPlayerIndex = VRPawn.getLocalPlayerIndex(),
    type = BallType,
});
--log("Spawning Ball")

Tee:spawnBallWithParameters(Parameters):andThen(function(Disc: DiscEntity)
    --log("Ball Now Spawned On Client")
    -- Disc:SetEnableHUDIndicator2(true)
    Disc:setEnableHUDIndicator(true)
    SpawnedBalls[Tee] = Disc
    MyBall = Disc

    -- When the ball goes to sleep/rest. Lift it up 10 units slowly
    Disc.onGoToSleep.Listen(function()
        --log("onGoToSleep")
        Disc.addSmoothOffset(Disc:getGravityDirection() * -10)
    end)
end)
```

BallSpawnParameters.new({ ... })

C++ struct `FBallSpawnParameters`, keys in lowerCamel.

targetPlayerIndex = VRPawn.getLocalPlayerIndex()

Each client spawns its own personal ball.

Tee:spawnBallWithParameters(Parameters):andThen(...)

A promise of the spawned `DiscEntity`.

Disc.onGoToSleep.Listen(...)

An event on the ball, fired when it comes to rest.

Disc:getGravityDirection() * -10

Vector times number. (`addSmoothOffset` is a `BallSpawnerComponent` method in the dump, not a `DiscEntity` one, so this call can't be confirmed from the dump.)

From the game: a state machine driven by timers (ScraprunPrime)

game-scripts/ScraprunPrime_gamemode.luau, lines 362-370

```
function SwitchState(state: number)
    GameStateManager:updateGameState(state)
    currentState = state
    stateMachineOnEnters[currentState].onEnter()
    local length = stateMachineOnEnters[currentState].basicTimerLength
    if length ~= nil then
        BasicTimer:start(length)
    end
end
```

game-scripts/ScraprunPrime_gamemode.luau, lines 757-764

```
function BeginPlay()
    BasicTimer.finished.Listen(BasicTimerFinished)
    GameTimer.onCountdownEnd.Listen(OnTimerCountdownFinished)
    GameTimer.onTimerStopped.Listen(OnTimerFinished)
    GameTimer.enterGameState.Listen(SwitchState)
    KillerTeamChanger.teamSizeDecreased.Listen(PlayerLeftTeam)
    RunnerTeamChanger.teamSizeDecreased.Listen(PlayerLeftTeam)
    StartButton.onButtonPressEvent.Listen(OnStartGameButtonPress)
```

`BasicTimer` is a `TimerComponent`. `BasicTimer:start(length)` starts it, and `BasicTimer.finished` fires when it runs out. The script keeps a table of states, and each state has an `onEnter` function, an optional `nextState` and an optional timer length.

From the game: passing data to the server (TKBGolf)

game-scripts/TKBGolf_ServerCourseLogic.luau, lines 108-116

```
EventComponent.serverEventData.Listen(function (Data: any)
    -- --log('Handling Event: ${Data.EventType}')
    if EventHandlers[Data.EventType] then
        -- --log("Handling Event")
        EventHandlers[Data.EventType](Data)
    else
        -- --log("Event not handled")
    end
end)
```

The course script (on every client) calls `MainEventRepeater:triggerEventWithData({ EventType = ..., ... })`, and the server-side scoreboard script receives the same table through `EventComponent.serverEventData`. Any Luau table can be sent this way (the `Ref` type).

10 Setting up VS Code

- 1 **Install VS Code** from `code.visualstudio.com`.
- 2 **Install the extension.** Open Extensions (`Ctrl+Shift+X`), search **Luau Language Server** by *JohnnyMorganz* (`JohnnyMorganz.luau-lsp`) and click Install.
- 3 **Open the kit folder.** `File` → `Open Folder...` and choose `SpecEditor\luau` itself. The settings in `.vscode/settings.json` use paths relative to this folder.
- 4 **Check it works.** Open `examples/01_hello.luau` and hover `log`: you should see `log(...: any): ()`. Type `VRPawn`, and the static functions appear.
- 5 **Write your script** as a new `.luau` file in `Documents\RigelScripts` (the editor's **Open folder in VS Code** button opens it with these settings), then attach it (chapter 2). Saving the file re-sends it.

What the settings do

`.vscode/settings.json`

```
{
  "luau-lsp.platform.type": "standard",
  "luau-lsp.sourcemap.enabled": false,
  "luau-lsp.types.definitionFiles": { "@rigel": "types/rigel.d.luau" },
  "luau-lsp.ignoreGlobs": ["game-scripts/**"]
}
```

- `platform.type: "standard"` makes this plain Luau, with no Roblox globals.
- `types.definitionFiles` loads the game API. The setting is a map from a name to a file, as the current extension expects.
- `ignoreGlobs` hides problems in the dumped game scripts, which call a few things the definitions can't know about.

`.luaurc`

```
{
  "languageMode": "strict",
  "lint": { "*": true, "FunctionUnused": false },
  "lintErrors": false,
  "typeErrors": true
}
```

`FunctionUnused` is off because nothing in the file calls `BeginPlay` and `EndPlay` (the game does).

What you get

- Completion after `obj:` (methods) and `obj.` (events, properties), with parameter hints.
- Red underlines for misspelt methods, wrong argument types and wrong argument counts.

- Typed event callbacks: in `Cup.onBallInCup_Multicast.Listen(function(cup, score, disc) ...)` the editor knows `score` is a number.

Definition syntax

`rigel.d.luau` uses `declare extern type Name with ... end`, the syntax current `luau-lsp` releases expect (checked with `luau-lsp 1.70`). Older releases used `declare class` and newer ones reject it. If an old extension shows "Failed to read definitions file", update it.

Regenerating

Run `python generate_defs.py` in the folder after the SDK dump changes, then *Luau: Restart Language Server* from the command palette.

11 Troubleshooting

Where the output goes

Every machine writes its own log. On a PC client it is `%LOCALAPPDATA%\A2\Saved\Logs\A2*.log` (the newest is `A2.log`; older runs are `A2_2.log` and so on). The server writes its own log on the server machine.

You wrote	The log shows
<code>log("hi")</code>	<code>LogLuau: Display: [MyScript.luau]: hi</code>
<code>warn("hi")</code>	<code>LogLuau: Warning: [MyScript.luau]: hi</code>

Search for `[MyScript.luau]` to see only your script's lines.

Errors pop up in the editor

When one of your scripts hits an error on any machine, the editor shows a popup with the file, the line, what went wrong in plain words and how to fix it, plus an **Open in VS Code** button that jumps to the line.

Errors can't crash the game

The server adds a small safety net to the end of every script: `BeginPlay`, `EndPlay` and `Tick` run inside `pcall`, and the game already catches errors in event listeners and timers. A mistake stops that one function and is reported; the object, the server and every player keep running. (The safety net is added after your last line, so line numbers in errors match your file.)

Nothing appears

- **Compile errors.** The editor checks every script for typos before sending it. If there's a typo, a popup names the line and what's wrong, and the script isn't sent. VS Code also shows syntax errors as you type. Then search the log for your script name and for `LogLuau` lines of Warning or Error verbosity. The exact error format hasn't been captured yet.
- **Only top-level code ran.** Check that `BeginPlay` is a global function with that exact spelling.
- **An early error in BeginPlay.** A runtime error stops the rest of the function. A `nil` dependency (no references wiring) is the usual cause, so check for it and log it, as the examples do.
- **Wrong machine.** Server-side events (`onOverlapByPlayerServer`) only fire on the server, so look in the server log for those.

The object changed after I attached a script

That's expected. Attaching **rebuilds the object** with the new component entry. Any runtime state it had is lost, and the script starts again from `BeginPlay`. Reloading a level does the same.

Something runs twice (or once per player)

The script runs on every machine (chapter 1). Guard work that should happen once: do authoritative work in server events, and per-player work in client-side events or behind a `VRPawn.getLocalPlayerIndex()` check.

VS Code problems

Symptom	Fix
"Unknown global 'log'", no completions	Open <code>SpecEditor\luau</code> as the folder (not the repo root), then <i>Luau: Restart Language Server</i> .
"Failed to read definitions file"	Update the Luau Language Server extension, or re-run <code>generate_defs.py</code> .
"Key 'Foo' not found in external type"	That method isn't in the class dump for that type. Check spelling (lowerCamel) and Appendix A.
"cannot be compared with ~= / =="	Cast to an optional type first: <code>(Door :: PhysicalComponent?) == nil</code> .
"Unknown require"	<code>require("OtherScript")</code> resolves inside the game, not on disk.

12 Limitations

Read this before building something big

The kit combines verified facts, game scripts and a class dump. They aren't equally reliable.

Topic	Status
Attaching, compiling and running on every machine; <code>Log</code>	Verified in Rigel (server, editor client, unmodded client).
Saving scripts with levels, re-attaching on load	Verified as editor behaviour.
<code>LuauClock</code> , <code>Promise</code> , events, <code>Vector</code> , <code>VRPawn.*</code> , <code>Quests.*</code> , <code>Gamemode</code>	Used by the game's own scripts, so they exist in the game's Luau environment. Not yet exercised from an editor-attached script.
Method lists in <code>rigel.d.luau</code>	Generated from the class dump. Every UFunction of every component is listed, but not every one is necessarily exposed to Luau or safe to call (some are network RPCs or engine callbacks such as <code>overlapBegin</code>). Prefer methods the game scripts call. Test in a local server first.
Properties	Public simple-typed fields are listed (the scripts read e.g. <code>disc.rollbackRecentPlayerHit</code>), but whether a particular field is readable or writable from Luau is unknown.
External Dependencies / references	Wired by dragging from the Outliner onto a slot. The target must be in the same gamemode area as the scripted object.
Blueprint-backed types, PascalCase spellings	Inferred from the game scripts (e.g. <code>GolfTeeComponent</code> → <code>BP_GolfTee_C</code>).
Return values	Object returns are typed as never-nil for convenience, but at runtime they can be <code>nil</code> (e.g. <code>getSpawnedBall()</code> before a ball exists). Check before use.
The game scripts themselves	They contain a few calls the dump doesn't back (<code>Disc:addSmoothOffset</code> on a <code>DiscEntity</code> , <code>resetBall</code> with extra arguments). Don't copy those as-is.

Safe habits

- Test on a local server with a mock or second client before a live station. Never try a new script on a server with players on it.
- Start with `log` lines around every call you haven't used before.
- Check every dependency for `nil`, and give every loop a stop condition.

Appendix: component and class reference

Every class in `rigel.d.luau`, with up to six key methods (engine callbacks and network RPCs left out), and its first few events. Call methods with `:` and listen to events with `.name.Listen(fn)`. The full lists, with C++ declarations, are in the definitions file.

LuauBehavior

C++ ULuauBehavior · SandboxEngine · LuauBehavior component

- `:isNetworkReady(): boolean`
- Events:** *none*

A2Object extends LuauBehavior

C++ UA2Object · SandboxEngine · LuauBehavior component

- `:getGamePosition(): Vector`
- Events:** *none*

ArenaSignUpComponent extends LuauBehavior

C++ UArenaSignUpComponent · A2 · LuauBehavior component

- `:getTicketManager(): ()`
- Events:** *none*

BallSpawnerComponent extends LuauBehavior

C++ UBallSpawnerComponent · A2 · LuauBehavior component

- `:addSmoothOffset(offset: Vector): ()`
- `:beginListeningForBallGrounded(): ()`
- `:checkForDeletedBalls(): ()`
- `:checkIfBallGroundedUponHit(hit: any, oldVelocity: Vector): ()`
- `:destroyBall(entity: DiscEntity): ()`
- `:discWasHit(): ()`
- *+ 22 more in rigel.d.luau*
- Events:** `.discWasSpawned`, `.discBeingDestroyed`, `.discWasSpawnedWithDiscArg`, `.onBallGrounded`, `.playerHitLuau`

BallLauncherComponent extends BallSpawnerComponent

C++ UBallLauncherComponent · A2 · LuauBehavior component

- `:grabBlock(blockGrabbed: any): ()`
- `:onBallThrown(grabComponent: any, throwVelocity: Vector, throwAngularVelocity: Vector): ()`
- `:onGoal(): ()`
- `:releaseBlock(): ()`
- `:setPlayerTarget(inPlayerTarget: any): ()`
- Events:** `.onBallLaunched`

BasicButtonComponent extends LuauBehavior

C++ UBasicButtonComponent · A2 · LuauBehavior component

- `:luaDisableButton(): ()`
- `:luaEnableButton(): ()`
- `:setButtonText(newText: string): ()`
- `:setOnCooldown(newBool: boolean): ()`
- Events:** `.onButtonPressEvent`

DataTableComponent extends LuauBehavior

C++ UDataTableComponent · A2 · LuauBehavior component

- `:beginTransaction(): ()`
- `:clearCellBackgroundColor(text: LinearColor, row: number, col: number): ()`
- `:clearCellTextColor(text: LinearColor, row: number, col: number): ()`
- `:endDataTransaction(): ()`
- `:getIndex(test: string): number`
- `:getTest(): any`
- *+ 7 more in rigel.d.luau*
- Events:** *none*

DriftballControlPanelComponent extends LuauBehavior

C++ UDriftballControlPanelComponent · A2 · LuauBehavior component

- `:practiceButtonOverlappedOnServer(): ()`
- `:practiceModeButtonEmitNetEvent(): ()`
- `:restartButtonEmitNetEvent(): ()`
- `:restartButtonOverlappedOnServer(): ()`
- Events:** `.restartButtonHit`, `.practiceModeButtonHit`

EffectVolume extends LuauBehavior

C++ UEffectVolume · A2 · LuauBehavior component

- `:checkOverlappingComponents(): ()`
- `:onOverlapBegin(overlappedActor: any, otherActor: any): ()`
- `:onOverlapEnd(overlappedActor: any, otherActor: any): ()`
- Events:** *none*

EventComponent extends LuauBehavior

C++ UEventComponent · A2 · LuauBehavior component

- `:eventCompleteWithDiscArgument(disc: DiscEntity): ()`
- `:getLocalPosition(): Vector`
- `:triggerEventWithData(ref: Ref): ()`
- `:triggerEventWithInt(integerArg: number): ()`
- `:triggerMulticastEventWithData(ref: Ref): ()`
- Events:** `.serverEvent`, `.serverEventData`, `.on_MulticastEventData`, `.eventCompleted`, `.eventCompleteWithInt`

GameStateManagerComponent extends LuauBehavior

C++ UGameStateManagerComponent · A2 · LuauBehavior component

- `:ballHitEvent_Multicast(i: number): ()`
- `:ballHitEvent_Server(i: number): ()`
- `:callBallHitsStartTrigger(): ()`
- `:skipGameStateTimer(): ()`
- `:updateGameState(newState: number): ()`
- Events:** `.skipGameState`, `.forceBallHitStartTrigger`, `.onGameStateChanged`, `.getArenaWalls`, `.ballHitEvent + 1 more`

GameTimeComponent extends LuauBehavior

C++ UGameTimeComponent · A2 · LuauBehavior component

- `:getCountdownMinutes(): string`
- `:getCountdownSeconds(): string`
- `:getGameMinutes(): string`
- `:getGameSeconds(): string`
- `:getGameTime(): string`
- `:getMsInGameClock(): number`
- *+ 14 more in rigel.d.luau*
- Events:** `.onGetGameTime`, `.onGetSoundManager`, `.onGetCountdownTime`, `.onTimerStopped`, `.onCountdownEnd + 1 more`

GamemodeLoaderComponent extends LuauBehavior

C++ UGamemodeLoaderComponent · A2 · LuauBehavior component

- *no methods of its own*
- Events:** *none*

GoalComponent extends LuauBehavior

C++ UGoalComponent · A2 · LuauBehavior component

- `:enableGoal(enable: boolean): ()`
- `:getForwardVectorLuau(): Vector`
- `:getPosition(): Vector`
- `:resetColor(bResetBlocks: boolean): ()`
- `:setColor(goalColor: LinearColor): ()`
- `:setColorByIndex(teamIndex: number): ()`
- *+ 6 more in rigel.d.luau*
- Events:** `.blueprintOnEffectOn`, `.blueprintOnEffectOff`, `.setGoalieBlocks`, `.setNeutralGoalieBlocks`, `.playSound + 5 more`

GoalieTrainingComponent extends LuauBehavior

C++ UGoalieTrainingComponent · A2 · LuauBehavior component

- :endBecausePlayerLeft(): ()
- :getRelevantPawn(): VRPawn
- :getSlot(): ()
- :luaDisable(): ()
- :luaEnable(): ()
- :playerHasJoined(): ()
- + 4 more in *rigel.d.luau*

Events: .blueprintOnDiscSpawned, .blueprintOnDisabled, .blueprintOnEnabled, .blueprintEndBecausePlayerLeft, .blueprintGoalieDifficultyChanged +2 more

GolfCupComponent extends LuauBehavior

C++ UGolfCupComponent · A2 · LuauBehavior component

- :ballInCup(score: number, entity: DiscEntity): ()
- :setCupVisibility(show: boolean): ()
- :setEnabledArrowIndicator(enable: boolean): ()
- :setEnabledHUDIndicator(enable: boolean): ()
- :setOutlineVisibility(show: boolean): ()
- + 1 more in *rigel.d.luau*

Events: .onBallInCup_Server, .onBallInCup_Multicast, .onSetCupVisibility, .onSetOutlineVisibility, .onSetEnabledHUDIndicator +1 more

HandholdComponent extends LuauBehavior

C++ UHandholdComponent · A2 · LuauBehavior component

- :getGrabbableComponent(): any
- :setHandHoldColor(color: LinearColor): ()

Events: none

LevelSwapperComponent extends LuauBehavior

C++ ULevelSwapperComponent · A2 · LuauBehavior component

- no methods of its own

Events: none

LuauBlueprintComponent extends LuauBehavior

C++ ULuauBlueprintComponent · SandboxEngine · LuauBehavior component

- no methods of its own

Events: .fLoadInInitialPersistents, .fLoadInGameDataFromPersistents, .fBeginGamemode, .fSetup

NetVarModifierComponent extends LuauBehavior

C++ UNetVarModifierComponent · A2 · LuauBehavior component

- no methods of its own

Events: none

NetworkRepeater extends LuauBehavior

C++ UNetworkRepeater · A2 · LuauBehavior component

- no methods of its own

Events: none

PasserSpawnerComponent extends LuauBehavior

C++ UPasserSpawnerComponent · A2 · LuauBehavior component

- :reachBegin(overlappedComponent: any, otherActor: any, otherComp: any, otherBodyIndex: number, fromSweep: boolean, sweepResult: any): ()
- :reachEnd(overlappedComponent: any, otherActor: any, otherComp: any, otherBodyIndex: number): ()
- + 2 more in *rigel.d.luau*

Events: none

PhysicalComponent extends LuauBehavior

C++ UPhysicalComponent · A2 · LuauBehavior component

- :disableCollisionLua(): ()
- :getLocalPosition(): Vector
- :hideLua(): ()
- :hideLuaClientside(): ()
- :setDefaultCollision(): ()
- :setTriggerCollision(): ()
- + 5 more in *rigel.d.luau*

Events: .onHitByDisc, .onOverlapByDisc, .onOverlapByPlayer, .onOverlapByDiscSimple, .onOverlapEndByDiscSimple +5 more

ProgressionButtonComponent extends LuauBehavior

C++ UProgressionButtonComponent · A2 · LuauBehavior component

- :onPressed(): ()

Events: none

ProgressionKioskComponent extends LuauBehavior

C++ UProgressionKioskComponent · A2 · LuauBehavior component

- no methods of its own

Events: none

RedCoinTimedQuestComponent extends LuauBehavior

C++ URedCoinTimedQuestComponent · A2 · LuauBehavior component

- :onBeforeEnterCoinEditMode(): ()
- :onEnterCoinEditMode(): ()
- :onExitCoinEditMode(): ()
- :onThrustersStateChanged(pawn: VRPawn, isOn: boolean): ()

Events: none

RockWallQuestComponent extends LuauBehavior

C++ URockWallQuestComponent · A2 · LuauBehavior component

- no methods of its own

Events: .onCleanupEvent, .onEndInspectionEvent, .onQuestEndEvent, .onQuestStartEvent, .onServerDataChangedEvent +1 more

ScoreComponent extends LuauBehavior

C++ UScoreComponent · A2 · LuauBehavior component

- :blueTest(): ()
- :decrementScoreBlue(): ()
- :decrementScoreRed(): ()
- :getTeamPoints(teamIndex: number): number
- :incrementScore(goalInfo: GoalInfo): ()
- :incrementScoreBlue(): ()
- + 5 more in *rigel.d.luau*

Events: .onScored, .updateFastestGoal, .updateLongestGoal, .updateBouncesGoal, .updatePassesGoal

ShootingTrainingComponent extends LuauBehavior

C++ UShootingTrainingComponent · A2 · LuauBehavior component

- :endBecausePlayerLeft(): ()
- :getRelevantPawn(): VRPawn
- :getSlot(): ()
- :luaDisable(): ()
- :luaEnable(): ()
- :playerHasJoined(): ()
- + 4 more in *rigel.d.luau*

Events: .blueprintOnDiscSpawned, .blueprintOnDisabled, .blueprintOnEnabled, .blueprintEndBecausePlayerLeft, .blueprintShootingDifficultyChanged +2 more

TeamBasedComponent extends LuauBehavior

C++ UTeamBasedComponent · A2 · LuauBehavior component

- no methods of its own

Events: .teamColorsChanged

TeamChangeComponent extends LuauBehavior

C++ UTeamChangeComponent · A2 · LuauBehavior component

- :getTeamSize(): number
- :onTeamSizesChanged(teamName: number, teamSize: number, maxSize: number): ()
- :onTicketManagerRemoved(): ()
- :onTicketManagerSpawned(manager: any): ()
- :populateVRPawTeamChangerArray(): ()
- :removeSelfFromVRPawTeamChangersArray(): ()
- + 4 more in [rigel.d.luau](#)

Events: .teamSizeIncreased, .teamSizeDecreased

TextComponent extends LuauBehavior

C++ UTextComponent · A2 · LuauBehavior component

- no methods of its own

Events: none

TicketManagerSpawnerComponent extends LuauBehavior

C++ UTicketManagerSpawnerComponent · A2 · LuauBehavior component

- :getSpawnedTicketManager(): any
- :killTicketManager(): ()
- :spawnTicketManager(newMaximumTeamSizes: any, newTeamColors: any): ()

Events: .ticketManagerSpawned

TimerComponent extends LuauBehavior

C++ UTimerComponent · A2 · LuauBehavior component

- :start(length: number): ()
- :startCountUpFromZero(): ()
- :stopAndResetTimer(): ()

Events: .finished, .finished2

ToggleableComponent extends LuauBehavior

C++ UToggleableComponent · A2 · LuauBehavior component

- :luaDisable(): ()
- :luaEnable(): ()

Events: .blueprintOnDisabled, .blueprintOnEnabled

TriggerComponent extends LuauBehavior

C++ UTriggerComponent · A2 · LuauBehavior component

- no methods of its own

Events: none

DiscEntity

C++ ADiscEntity · A2 · used by the game scripts

- :ballStolen(ballLocation: Vector): ()
- :forceUpdate(): ()
- :freeze(hideOutline: boolean): ()
- :getCosmeticLoadout(): any
- :getDiscBounces(): number
- :getDiscPasses(): number
- + 32 more in [rigel.d.luau](#)

Events: .onGoToSleep, .playerHit

VRPaw

C++ AVRPAW · A2 · used by the game scripts

- :addImpulse(move: Vector, bAddToPushOffMomentum: boolean): ()
- :addItemToInventoryByGameplayTag(inItemType: any, inGameplayTag: any): number
- :addItemToInventoryById(inItemId: string): number
- :addTicketSystemCooldown(seconds: number): ()
- :arenaSignUpReadyAlert(): ()
- :attemptTakeOwnershipProp(actor: any, previousOwner: any, lastSeenTimestamp: number, isHolding: boolean, isLeftHandHolding: boolean): ()
- + 164 more in [rigel.d.luau](#)

Events: .onHighFive, .onBeforeDestroy, .flaserStateUpdatedForBlueprint, .initialPlayerEnteredArena, .lastPlayerLeftArena +14 more

ModuleStateLuaAPI

C++ UModuleStateLuaAPI · SandboxEngine · used by the game scripts

- :broadcastEvent(eventName: string): ()
- :broadcastEventBool(eventName: string, value: boolean): ()
- :broadcastEventNumber(eventName: string, value: number): ()
- :broadcastEventString(eventName: string, string: string): ()
- :emitStatEventForAllPlayers(string: string, i: number): ()
- :emitStatEventForAllPlayersOnTeam(teamIndex: number, string: string, i: number): ()
- + 14 more in [rigel.d.luau](#)

Events: .onConfigChanged

GameStateManagerBlueprintComponent

C++ ABP_GameStateManager_C · BP_GameStateManager · blueprint-backed component used by a game script

- :getSetArenaWall(): ()
- :getSlot(): any
- :playClearEvent(): ()
- :playEspeciallyGoodShotEvent(): ()
- :playSaveEvent(): ()
- :playShotEvent(): ()
- + 3 more in [rigel.d.luau](#)

Events: .gameCountdownStarting

DriftballControlPanelBlueprintComponent

C++ ABP_DriftballControlPanel_C · BP_DriftballControlPanel · blueprint-backed component used by a game script

- :bIsRestartableChanged(value: boolean): ()
- :handScannerEndOverlapped(overlappedComponent: any, otherActor: any, otherComp: any, otherBodyIndex: number): ()
- :handScannerOverlapped(overlappedComponent: any, otherActor: any, otherComp: any, otherBodyIndex: number, bFromSweep: boolean, sweepResult: any): ()
- :playLockSound(): ()
- :playPracticeButtonSound(): ()
- :playRestartButtonSound(): ()
- + 3 more in [rigel.d.luau](#)

Events: none

TrapSpinnerComponent

C++ ABP_Trap_Spinner_C · BP_Trap_Spinner · blueprint-backed component used by a game script

- :reverseDirection(): ()
- :setCanMove(bNewCanMove: boolean): ()
- :setRevolutionsPerCycle(setRevolutionsPerCycle: number): ()
- :trapActivate(): ()
- :trapDeactivate(): ()
- + 3 more in [rigel.d.luau](#)

Events: none

TrapRotatingPlatformComponent

C++ ABP_Trap_RotatingPlatform_C · BP_Trap_RotatingPlatform · blueprint-backed component used by a game script

- :reverseDirection(): ()
- :setCanMove(bNewCanMove: boolean): ()
- :setRevolutionsPerCycle(setRevolutionsPerCycle: number): ()
- :trapActivate(): ()
- :trapDeactivate(): ()
- + 3 more in [rigel.d.luau](#)

Events: none

SlidingPlatformComponent

C++ ABP_SlidingPlatform_C · BP_SlidingPlatform · blueprint-backed component used by a game script

- :bIsBlackChanged(value: boolean): ()
- :setCanMove(bNewCanMove: boolean): ()
- :setIsBlack(bNewIsBlack: boolean): ()
- :trapActivate(): ()
- :trapDeactivate(): ()

Events: none

DeathrunResetTeleporterComponent

C++ ABP_DeathrunResetTeleporter_C · BP_DeathrunResetTeleporter · blueprint-backed component used by a game script

- `:onRep(value: boolean): ()`
- `:toggleRegularResetTeleporter(): ()`
- `:toggleSpleefTeleporter(): ()`

Events: *none*

aa_se_BP_TeleporterComponent

C++ ALE_BP_Teleporter_C · LE_BP_Teleporter · blueprint-backed component used by a game script

- `:activateTeleporter(): ()`
- `:deActivateTeleporter(): ()`
- `:setTeleportLocationRelativeToModuleSlot(locationRelativeToModuleSlot: Vector): ()`
- `:teleporterActiveNetvarChanged(value: boolean): ()`
- `:teleportLocationChanged(value: Vector): ()`

Events: *none*

GolfTeeComponent

C++ ABP_GolfTee_C · BP_GolfTee · blueprint-backed component used by a game script

- `:ballScored(playerName: string): ()`
- `:deactivateBall(): ()`
- `:getActorRef(): any`
- `:getActorTags(): any`
- `:resetBallToTee(): ()`
- `:resetStoredScore(playerId: number): ()`
- + 12 more in *rigel.d.luau*

Events: `.onManualStart`, `.onManualDespawn`, `.onManualReset`, `.onManualEnd`, `.onTeeActivated` + 2 more

B Appendix: global tables

Static functions and constructors available as globals, in addition to `log`, `warn`, `Logging`, `Promise`, `Vector`/`Vec3`, `LinearColor` and `Enum.TextJustify`.

Global	Members
VRPawn	<code>getLocalInstance(): () -> VRPawn</code> <code>getLocalInstanceWithWorld(): () -> VRPawn</code> <code>getLocalPlayer(): () -> VRPawn</code> <code>getLocalPlayerIndex(): () -> number</code> <code>getLocalVRPawn(): () -> VRPawn</code> <code>getPlayer(id: number) -> VRPawn</code> <code>getPlayerByID(id: number) -> VRPawn</code> <code>getPlayerName(id: number) -> string</code> <code>getPositionByID(id: number) -> Vector</code> <code>getTeamIndexByID(id: number) -> number</code>
Quests	<code>complete(playerId: number, questID: string) -> ()</code> <code>event(playerId: number, eventID: string) -> ()</code> <code>eventNumber(playerId: number, eventID: string, number: number) -> ()</code>
LuauClock	<code>createTimer(betweenTime: number, callback: () -> ()) -> number</code> <code>getTime(): () -> number</code> <code>timeout(seconds: number) -> Promise<()></code>
BallSpawnParameters	<code>new: (fields: { position: Vector?, isPersonalBall: boolean?, targetPlayerIndex: number?, type: string? }) -> BallSpawnParameters</code>
GoalInfo	<code>new: (fields: { [string]: any }) -> GoalInfo</code>
CellStyle	<code>new: (fields: { [string]: any }) -> CellStyle</code>
LayoutSettings	<code>new: (fields: { [string]: any }) -> LayoutSettings</code>
LayoutGap	<code>new: (targetIndex: number, gap: number) -> LayoutGap</code>
StyleOverRange	<code>new: (style: CellStyle, startRow: number, startCol: number, endRow: number, endCol: number) -> StyleOverRange</code>
Gamemode	<code>instance of ModuleStateLuaAPI</code>